

Оптимизация

Лабораторная работа № 8

Шулуужук А. В.

26 ноябрь 2025

Российский университет дружбы народов, Москва, Россия

Основная цель работы — освоить пакеты Julia для решения задач оптимизации.

Выполнение лабораторной работы

```
using JuMP
using GLPK

model = Model{GLPK.Optimizer}
# Определение переменных x, y и граничных условий для них:
@variable(model, x >= 0)
@variable(model, y >= 0)
# Определение ограничений модели:
@constraint(model, 6x + 8y >= 100)
@constraint(model, 7x + 12y >= 120)
# Определение целевой функции:
@objective(model, Min, 12x + 20y)
# Вызов функции оптимизации:
optimize!(model)
# Определение причины завершения работы оптимизатора:
termination_status(model)
# Демонстрация первичных результирующих значений переменных x и y:
@show value(x);
@show value(y);
# Демонстрация результата оптимизации:
@show objective_value(model);

value(x) = 14.99999999999993
value(y) = 1.250000000000047
objective_value(model) = 205.0
```

Рис. 1: Линейное программирование

Векторизованные ограничения и целевая функция оптимизации

```
# Определение объекта модели с именем vector_model:
vector_model = Model(GLPK, Optimizer)
# Определение начальных данных:
A = [ 1 1 9 5;
      3 5 0 8;
      2 0 6 13]
b = [7; 3; 5]
c = [1; 3; 5; 2]
# Определение Вектора переменных:
@variable(vector_model, x[1:4] >= 0)
# Определение ограничений модели:
@constraint(vector_model, A * x .== b)
# Определение целевой функции:
@objective(vector_model, Min, c' * x)
# Вызов функции оптимизации
optimize!(vector_model)

# Определение причины завершения работы оптимизатора:
termination_status(vector_model)

OPTIMAL::TerminationStatusCode = 1

# Демонстрация результата оптимизации:
@show objective_value(vector_model);

objective_value(vector_model) = 4.9230769230769225
```

Рис. 2: Векторизованные ограничения и целевая функция оптимизации

```
# Контейнер для хранения данных об ограничениях на количество потребляемых калорий, белков и соли:
category_data = JuMP.Containers.DenseAxisArray{
    [1800 2200;
     91 Inf;
     0 65;
     0 1779],
    [{"calories", "protein", "fat", "sodium"},
     {"min", "max"}]

# массив данных с наименованиями продуктов:
foods = ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]

# Массив стоимостей продуктов:
cost = JuMP.Containers.DenseAxisArray{[2.49, 2.89, 1.50, 1.89, 2.89, 1.99, 2.49, 0.89, 1.59], foods}

# Массив данных о содержании калорий, белков, жиров и соли в продуктах питания:
food_data = JuMP.Containers.DenseAxisArray{
    [410 24 26 730;
     420 32 10 1190;
     560 20 32 1800;
     380 4 19 270;
     320 12 10 930;
     320 15 12 820;
     320 31 12 1230;
     100 8 2.5 125;
     330 8 10 180],
    foods,
    [{"calories", "protein", "fat", "sodium"}]

2-dimensional DenseAxisArray{Float64,2,...} with index sets:
  Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
  Dimension 2, [{"calories", "protein", "fat", "sodium"}]
And data, a 9x4 Matrix{Float64}:
410.0  24.0  26.0  730.0
420.0  32.0  10.0  1190.0
560.0  20.0  32.0  1800.0
380.0   4.0  19.0  270.0
320.0  12.0  10.0  930.0
320.0  15.0  12.0  820.0
320.0  31.0  12.0  1230.0
100.0   8.0   2.5  125.0
330.0   8.0  10.0  180.0
```

Рис. 3: Оптимизация рациона питания

```
# Определение объекта модели с именем model:
model = Model(GLPK.Optimizer)
# Определим массив:
categories = ["calories", "protein", "fat", "sodium"]

# Определение переменных:
@variables(model, begin
category_data[c, "min"] <= nutrition[c = categories] <= category_data[c, "max"]
# Сколько покупать продуктов:
buy[foods] >= 0
end)

# Определение целевой функции:
@objective(model, Min, sum(cost[f] * buy[f] for f in foods))

@constraint(model, [c in categories],
sum(food_data[f, c] * buy[f] for f in foods) == nutrition[c])

# Вызов функции оптимизации:
JuMP.optimize!(model)
term_status = JuMP.termination_status(model)

OPTIMAL::TerminationStatusCode = 1

hcat(buy.data, JuMP.value.(buy.data))

9x2 Matrix{AffExpr}:
buy[hamburger]  0.6045138888888888
buy[chicken]    0
buy[hot dog]    0
buy[fries]      0
buy[macaroni]   0
buy[pizza]      0
buy[salad]      0
buy[milk]       6.9701388888888935
buy[ice cream]  2.5913194444444441
```

Рис. 4: Оптимизация рациона питания

```
using DelimitedFiles
using CSV

# Считывание данных:
passportdata = readdlm(joinpath("passport-index-matrix.csv"),',')
# Задаём переменные:
cntr = passportdata[2:end,1]
vf = (x -> typeof(x) == Int64 || x == "VF" || x == "VOA" ? 1 : 0).(passportdata[2:end,2:end]);
# Определение объекта модели с именем model:
model = Model{GLPK.Optimizer}

# Переменные, ограничения и целевая функция:
@variable(model, pass[1:length(cntr)], Bin)
@constraint(model, [j=1:length(cntr)], sum( vf[i,j]*pass[i] for i in 1:length(cntr)) >= 1)
@objective(model, Min, sum(pass))

# Вызов функции оптимизации:
JuMP.optimize!(model)
termination_status(model)

# Просмотр результата:
print(JuMP.objective_value(model)," passports: ",join(cntr[findall(JuMP.value.(pass) .== 1)],", "))
```

63.0 passports: Afghanistan, Andorra, Argentina, Australia, Azerbaijan, Bahrain, Brunei, Cambodia, Cameroon, Canada, o, Djibouti, Equatorial Guinea, Eritrea, Fiji, Gabon, Georgia, Guinea, Guinea-Bissau, Hong Kong, Hungary, Indonesia, pan, Kuwait, Laos, Liberia, Libya, Macao, Madagascar, Malaysia, Maldives, Marshall Islands, Mauritania, Mauritius, M New Zealand, North Korea, Palestine, Papua New Guinea, Qatar, Saudi Arabia, Solomon Islands, Somalia, South Sudan, S e, Togo, Turkmenistan, United States, Uruguay, Vietnam

Рис. 5: Путешествие по миру

Портфельные инвестиции

```
# Считываем данные и размещаем их во фрейм:
T = DataFrame(XLSX.readtable("stock_prices.xlsx", "Sheet2"))

# Построение графика:
plot(T[1, :MSFT], label="Microsoft")
plot(T[1, :AAPL], label="Apple")
plot(T[1, :FB], label="FB")

# Данные о ценах на акции размещаем в матрице:
prices_matrix = Matrix(T)

# Вычисление матрицы доходности за период времени:
M1 = prices_matrix[1:end-1, :]
M2 = prices_matrix[2:end, :]

# Матрица доходности:
R = (M2 - M1) ./ M1

# Матрица рисков:
risk_matrix = cov(R)

# Проверка положительной определённости матрицы рисков:
isposdef(risk_matrix)

# Доход от каждой из компаний:
r = mean(R, dims=1)[1]

# Вектор инвестиций:
x = Variable(length(r))

Variable
size: (3, 1)
sign: real
vexity: affine
id: 331..731
```

Рис. 6: Портфельные инвестиции

```
x
Variable
size: (3, 1)
sign: real
vexity: affine
id: 722...903
value: [0.07740709795031023, 0.11606771003983785, 0.8065276266877958]

sum(x.value)

1.000002434677944

r'*x.value

1x1 adjoint(::Vector{Float64}) with eltype Float64:
 0.019947233108531883

x.value .* 1000

3x1 Matrix{Float64}:
 77.40709795031023
116.06771003983785
 806.5276266877958
```

Рис. 7: Портфельные инвестиции

```
# Считывание исходного изображения:
```

```
Kref = load("kham-small.jpg")
```



```
K = copy(Kref)
```

```
p = prod(size(K))
```

```
missingids = rand(1,p,400)
```

```
K[missingids] = RGBX{NOTB}(0.0,0.0,0.0)
```

```
Gray.(K)
```



Рис. 8: Восстановление изображения

Восстановление изображения

```
# Находим решение
solve!(problem, SCS.Optimizer)

-----
SCS v3.2.9 - Splitting Conic Solver
(c) Brendan O'Donoghue, Stanford University, 2012
-----
problem: variables n: 80090, constraints m: 159779
cones:    z: primal zero / dual free vars: 79689
          nuc: nuclear vars: 80090, nusize: 1
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
          alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
          max_iters: 100000, normalize: 1, rho_x: 1.00e-06
          acceleration_lookback: 10, acceleration_interval: 10
          compiled with openmp parallelization enabled
lin-sys:  sparse-direct-and-qdldl
          nnz(A): 159779, nnz(P): 0
-----
iter | pri res | dua res | gap  | obj  | scale | time (s)
-----
0 | 1.27e+02 | 1.17e+01 | 3.09e+03 | 1.53e+03 | 1.00e-01 | 3.04e-01

@show norm(float.(Gray.(Kref))-X.value)
@show norm(-X.value)
colorview(Gray, X.value)

norm(float.(Gray.(Kref)) - X.value) = 1.2349050767677132
norm(-X.value) = 124.33772235725975
```



Рис. 9: Восстановление изображения

Самостоятельная работа

```
using JUMP, GLPK

# Создаем модель
model = Model(GLPK.Optimizer)

# Определяем переменные
@variable(model, 0 <= x1 <= 10)
@variable(model, x2 >= 0)
@variable(model, x3 >= 0)

# Определяем ограничения
@constraint(model, -x1 + x2 + 3x3 <= -5)
@constraint(model, x1 + 3x2 - 7x3 <= 10)

# Определяем целевую функцию (максимизация)
@objective(model, Max, x1 + 2x2 + 5x3)

# Решаем задачу
optimize!(model)

# Выводим результаты
println("Статус решения: ", termination_status(model))
println("Оптимальное значение целевой функции: ", objective_value(model))
println("Оптимальные значения переменных:")
println("x1 = ", value(x1))
println("x2 = ", value(x2))
println("x3 = ", value(x3))

Статус решения: OPTIMAL
Оптимальное значение целевой функции: 19.0625
Оптимальные значения переменных:
x1 = 10.0
x2 = 2.1875
x3 = 0.9375
```

Рис. 10: Линейное программирование

Линейное программирование. Использование массивов

```
using JuMP, GLPK, LinearAlgebra
# Определяем данные в матричной форме
A = [-1 1 3;      # матрица коэффициентов ограничений
      1 3 -7]
b = [-5, 10]      # правые части ограничений
c = [1, 2, 5]      # коэффициенты целевой функции
lb = [0, 0, 0]      # нижние границы
ub = [10, Inf, Inf] # верхние границы
# Создаем модель
model = Model{GLPK.Optimizer}()
# Определяем переменные
@variable(model, lb[i] <= x[i] <= ub[i])
# Определяем ограничения в матричной форме
@constraint(model, A * x .<= b)
# Определяем целевую функцию
@objective(model, Max, dot(c, x))
# Решаем задачу
optimize!(model)
# Выводим результаты
println("Статус решения: ", termination_status(model))
println("Оптимальное значение целевой функции: ", objective_value(model))
println("Оптимальные значения переменных:")
for i in 1:3
    println("x$i = ", value(x[i]))
end
# Проверяем ограничения
println("\nПроверка ограничений:")
println("Ограничение 1: ", dot(A[1,:], value.(x)), " <= ", b[1])
println("Ограничение 2: ", dot(A[2,:], value.(x)), " <= ", b[2])

Статус решения: OPTIMAL
Оптимальное значение целевой функции: 19.8625
Оптимальные значения переменных:
x1 = 10.8
x2 = 2.1875
x3 = 0.9375

Проверка ограничений:
Ограничение 1: -5.0 <= -5
Ограничение 2: 10.8 <= 10
```

Рис. 11: Линейное программирование. Использование массивов

```
using Convex, SCS, LinearAlgebra, Random
# Устанавливаем seed для воспроизводимости
Random.seed!(123)
# Генерируем случайные данные
m, n = 10, 5 # размерности
A = randn(m, n)
b = randn(m)
println("Размер матрицы A: ", size(A))
println("Размер вектора b: ", size(b))
# Создаем переменную
x = Variable(n)
# Определяем задачу оптимизации - минимизация квадрата нормы
problem = minimize(sumsquares(A * x - b))
problem.constraints += x >= 0
# Решаем задачу
solve!(problem, SCS.Optimizer)
# Выводим результаты
println("\nСтатус решения: ", problem.status)
println("Оптимальное значение целевой функции: ", problem.optval)
println("Норма невязки: ", norm(A * evaluate(x) - b))
println("\nОптимальные значения переменных:")
println(evaluate(x))
# Проверяем неотрицательность решения
println("Минимальное значение x: ", minimum(evaluate(x)))
println("Все ли x >= 0: ", all(evaluate(x) .>= 0))

Размер матрицы A: (10, 5)
Размер вектора b: (10,)
```

Рис. 12: Выпуклое программирование


```
-----
SCS v3.2.9 - Splitting Conic Solver
(c) Brendan O'Donoghue, Stanford University, 2012
-----
problem: variables n: 6, constraints m: 17
cones:   1: linear vars: 5
         q: soc vars: 12, qsize: 1
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
          alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
          max_iters: 100000, normalize: 1, rho_x: 1.00e-06
          acceleration_lookback: 10, acceleration_interval: 10
          compiled with openmp parallelization enabled
lin-sys: sparse-direct-and-qdldl
          nnz(A): 57, nnz(P): 0
-----
iter | pri res | dua res | gap | obj | scale | time (s)
-----
0 | 3.60e+01 | 1.00e+00 | 5.08e+01 | -2.47e+01 | 1.00e-01 | 3.76e-03
200 | 5.69e-04 | 3.89e-05 | 4.69e-04 | 9.84e+00 | 4.73e-01 | 5.11e-03
-----
status: solved
timings: total: 5.11e-03s = setup: 1.76e-04s + solve: 4.94e-03s
          lin-sys: 9.43e-05s, cones: 1.16e-03s, accel: 1.16e-05s
-----
objective = 9.842321
-----

Статус решения:
[ Info: [Convex.jl] Compilation finished: 8.02 seconds, 627.658 MiB of memory allocated
ОПТИМАЛ
Оптимальное значение целевой функции: 9.842887895461078
Норма невязки: 3.1380664132915537

Оптимальные значения переменных:
[0.0003370128953110813, 0.3445801421087122, 0.0002877987167345401, 7.074048672363908e-5, 0.0005690182161444619]
Минимальное значение x: 7.074048672363908e-5
Все ли x >= 0: true
```

Рис. 13: Выпуклое программирование

Оптимальная рассадка по залам

Статус решения: OPTIMAL

Средний приоритет: 1.357

Распределение участников по секциям:

Секция 1: 220 участников (min: 180, max: 250)

Секция 2: 183 участников (min: 180, max: 250)

Секция 3: 220 участников (min: 180, max: 250)

Секция 4: 180 участников (min: 180, max: 250)

Секция 5: 197 участников (min: 180, max: 250)

Рекомендации для первых 5 участников:

Участник 1: секция 4 (приоритет: 1)

Доступные секции: [2, 4, 5] с приоритетами: [3, 1, 2]

Участник 2: секция 3 (приоритет: 1)

Доступные секции: [2, 3, 4] с приоритетами: [2, 1, 2]

Участник 3: секция 4 (приоритет: 1)

Доступные секции: [1, 4, 5] с приоритетами: [3, 1, 3]

Участник 4: секция 3 (приоритет: 1)

Доступные секции: [1, 3, 5] с приоритетами: [2, 1, 3]

Участник 5: секция 2 (приоритет: 1)

Доступные секции: [2, 3, 5] с приоритетами: [1, 2, 1]

Рис. 14: Оптимальная рассадка по залам

Статус решения: OPTIMAL

Максимальная выручка: 5000.0 рублей

Оптимальный план производства:

Раф кофе: 8.0 чашек

Капучино: 6.0 чашек

Общее количество чашек: 14.0

Использование ресурсов:

Зёрна: 500.0 гр. из 500 гр. (100.0%)

Молоко: 1840.0 гр. из 2000 гр. (92.0%)

Ванильный сахар: 40.0 гр. из 40 гр. (100.0%)

Проверка выручки: 5000.0 рублей

Рис. 15: План приготовления кофе

Выводы

В результате выполнения лабораторной работы были освоены специализированные пакеты для решения задач оптимизации